You said:

I want to code and verify how the rerank in RAG working. Can you suggest some good project to understand RAG pipline properly.

Worked for 16s

ChatGPT said:

If your goal is to understand **how reranking actually works inside a RAG (Retrieval-Augmented Generation) pipeline**, I recommend building a project where you can inspect every stage, run experiments, and measure whether reranking improves the final answer.

Given your experience in IT and your interest in understanding the internal workings of rerankers, I would avoid starting with a high-level framework that hides the implementation. Instead, build a small RAG system in Python where you implement retrieval, reranking, evaluation, and answer generation as separate components.

1. My recommended project: Build a RAG Reranking Laboratory

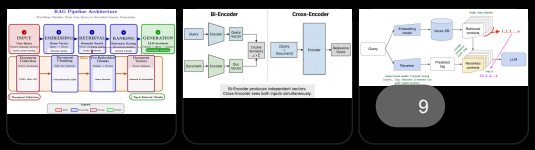
Project name: RAG Retrieval and Reranking Evaluation Lab

Objective: Build a working RAG pipeline that retrieves 50 document chunks, reranks them, selects the best 5, and passes them to an LLM to generate an answer. Then prove experimentally whether reranking improves retrieval quality.

You should be able to answer questions such as:

- Why did the retriever return these 50 documents?
- Why was a relevant document ranked 35th instead of first?
- How does a cross-encoder decide that one document is more relevant than another?
- Why does reranking move a document from position 35 to position 2?
- What happens when you change the number of retrieved documents from 20 to 100?
- How do recall, precision, MRR, and NDCG change after reranking?
- Does reranking improve the final answer, or does it merely change the document order?

The architecture you will build



The pipeline will look like this:

Plain text

```
                USER QUESTION
                      |
                      v
            Query Preprocessing
                      |
                      v
          +-----------------------+
          |  Document Collection  |
          |                       |
          |  500-2,000 documents  |
          +-----------------------+
                      |
                      v
            Document Chunking
                      |
                      v
             Embedding Model
                      |
                      v
```

```
Vector Database
      |
```

Public link copied
Anyone with the link can view this conversation. Learn more
Manage links

The important distinction is that **the re**

- The retriever searches a large co
- The reranker examines the query and each retrieved document more carefully to estimate their relevance.
- The LLM uses the selected documents to generate an answer.

Reranking cannot recover a relevant document that the initial retriever never retrieved. That is why you must measure retrieval quality before and after reranking separately.

2. What technology should you use?

I recommend keeping the first version simple and implementing the pipeline in Python without LangChain or LlamaIndex. You can introduce those frameworks later, after you understand what they abstract away.

Component	Recommended technology	Why
Programming language	Python	Easy to experiment with retrieval models
Document storage	JSON or JSONL files	Easy to inspect the source documents
Chunking	Python functions	Understand chunk size and overlap
Embeddings	Sentence Transformers	Learn how text becomes vectors
Vector search	NumPy cosine similarity initially	See exactly how retrieval scores are calculated
Optional vector database	FAISS	Experiment with efficient similarity search
Initial lexical retrieval	BM25 using rank-bm25	Compare keyword retrieval with semantic retrieval
Reranking	Sentence Transformers CrossEncoder	Inspect query-document relevance scores
Answer generation	Local LLM or an LLM API	Understand how retrieved context becomes an answer
Evaluation	Python, pandas, scikit-learn	Calculate and compare retrieval metrics

For your first reranker, use `cross-encoder/ms-marco-MiniLM-L6-v2`. It is a practical starting point for experimenting with cross-encoder reranking. Sentence Transformers provides a working retrieve-and-rerank example and evaluation utilities. Sbert +1

Documentation to keep handy:

- Sentence Transformers: Retrieve and Re-Rank
- Cross-Encoder Quickstart
- Cross-Encoder Evaluation

Important: You don't need a vector database or an LLM API in the first experiment. NumPy can handle a small collection, and the reranker can be tested independently. Add the other components incrementally.

3. The most important experiment: Prove that reranking works

This is the core of your project.

Imagine your document collection contains the following passages.

Document	Content	Relevance to the question
D1	A bi-encoder generates embeddings for queries and documents independently.	High

Document	Content	Relevance to the question
D2	A cross-encoder processes a query and a document together.	Very high
D3	A vector database stores embeddings for similarity search.	Medium
D4	RAG combines retrieved information with a language model.	Medium
D5	A cross-encoder uses interactions between query and document tokens to estimate relevance.	Very high
D6	Python supports object-oriented programming.	None

User query:

> How does a cross-encoder improve the ranking of documents in a RAG pipeline?

Suppose the initial retriever returns the documents in this order:

Initial rank	Document	Similarity score
1	D3	0.87
2	D1	0.83
3	D4	0.80
4	D2	0.77
5	D5	0.74

These scores and rankings are illustrative, not actual model results.

Notice the problem: D2 and D5 contain information directly relevant to the question, but the initial retriever ranks them fourth and fifth.

Now send the query and each retrieved document through the cross-encoder.

The model scores each query-document pair, and you sort the documents by those scores.

For example, the reranked results might look like this:

New rank	Document	Illustrative reranker score
1	D5	7.2
2	D2	6.8
3	D1	5.1
4	D3	3.4
5	D4	2.8

Again, these are hypothetical scores. Actual values depend on the model, its input formatting, and the documents.

The critical point is that the reranker has changed the order based on a different relevance-scoring mechanism.

A cross-encoder processes the query and document together, allowing attention to model interactions between their tokens. A bi-encoder instead computes their representations independently before comparing them. GitHub

What you need to investigate in the code

Don't stop after observing the new order. Investigate these questions:

1. **Input:** What exact tokens are supplied to the cross-encoder?
2. **Tokenization:** How are the query and document combined, and what happens when the input is too long?
3. **Model execution:** How does the Transformer process the combined input?
4. **Scoring:** What does the output score represent? Is it a probability or a raw model score?
5. **Sorting:** How does the program convert the scores into a new ranking?
6. **Selection:** What changes when you keep the top 3 instead of the top 5?
7. **Evaluation:** Did the relevant documents actually move up the ranking?

A crucial detail: **a reranker score is not automatically a calibrated probability of relevance.** For a typical MS MARCO cross-encoder, use the score to compare documents rather than assuming that a score of 7 means 70% relevance.

4. Build the project in five phases

I recommend completing these phases in order. Each phase should produce something you can run and verify before moving forward.

Phase 1: Build a retriever without a reranker

Goal: Understand why documents are retrieved in a particular order.

Create 100–500 sample document chunks about topics such as RAG, embeddings, vector databases, Transformer models, and Python.

Implement:

- Document loading.
- Chunking.
- Embedding generation.
- Cosine similarity.
- Top-k document retrieval.

For each query, print the following:

Plain text

```
Query: How does a cross-encoder work?

Rank 1: Document D12
Cosine similarity: 0.82

Rank 2: Document D45
Cosine similarity: 0.79

Rank 3: Document D08
Cosine similarity: 0.76
```

Your program should also record which documents are genuinely relevant, using manually labelled test data.

What you will learn: How a bi-encoder converts text into vectors and how vector similarity determines the initial ranking.

Phase 2: Add a cross-encoder reranker

Goal: Understand reranking at the model and code levels.

Install the required libraries:

Bash

```
pip install sentence-transformers numpy pandas scikit-learn
```

Then create `reranker.py`:

Python

```
from sentence_transformers import CrossEncoder
```

The first run downloads the model, so you need internet access and sufficient storage. Subsequent runs can use the cached model.

This implementation deliberately excludes the vector retriever. That allows you to verify the reranker independently before combining the two components.

Experiment: Change the query while keeping the documents fixed. Observe how the scores and ranking change.

Next, change one document's wording without changing its underlying meaning. Does its ranking change? This helps you understand both the strengths and limitations of relevance models.

Phase 3: Measure whether reranking improves retrieval

Goal: Move from observing model output to scientifically evaluating it.

Create a small labelled evaluation dataset:

Python

```
test_cases = [
    {
        "query": "How does a cross-encoder work?",
        "relevant_doc_ids": ["D2", "D4"]
    },
    {
        "query": "Why does RAG need a vector database?",
        "relevant_doc_ids": ["D1", "D5"]
    }
]
```

These labels are examples for your own test collection; you should verify them against the actual documents.

Now implement these metrics:

Metric	What it tells you
Recall@K	How many relevant documents were retrieved within the first K results?
Precision@K	What proportion of the first K results are relevant?
MRR@K	How high is the first relevant result ranked?
NDCG@K	How good is the ranking when relevance and position are considered?
Latency	How long does retrieval and reranking take?

The most important comparison

Run the same queries through two configurations:

Experiment A — Without reranking

```
Plain text
Query
  |
  v
Vector retrieval: Top 50
  |
  v
Select Top 5
```

Experiment B — With reranking

```
Plain text
Query
  |
  v
Vector retrieval: Top 50
  |
  v
Cross-encoder reranking: 50 documents
  |
  v
Select Top 5
```

Compare the metrics across both experiments.

One important rule: **use the same initial 50 retrieved documents in both experiments.** Otherwise, you cannot isolate the effect of reranking.

Also measure Recall@50 before reranking. That tells you whether the retriever found the relevant documents in the first place.

The Sentence Transformers documentation includes a reranking evaluator that can calculate metrics such as MRR, NDCG, and MAP. See its evaluation documentation.

Phase 4: Experiment with different retrieval strategies

Goal: Understand why production RAG systems often combine multiple retrieval methods.

Once the first three phases work, compare these configurations:

Experiment	Retrieval method	Reranker
A	BM25 keyword search	None
B	Dense vector search	None
C	Dense vector search	Cross-encoder
D	BM25 + dense hybrid retrieval	None
E	BM25 + dense hybrid retrieval	Cross-encoder

This will help you investigate a crucial question:

Does a better reranker compensate for a weak retriever?

Not completely. If the retriever misses a relevant document, the reranker cannot rank that document. Hybrid retrieval may improve the candidate set by combining exact keyword matching with semantic similarity.

For hybrid retrieval, you can explore Reciprocal Rank Fusion (RRF), which combines ranked lists from multiple retrieval methods.

For a reference implementation, explore the Sentence Transformers retrieve-and-rerank examples.

Phase 5: Complete the end-to-end RAG pipeline

Goal: Verify whether reranking improves the answers produced by the LLM.

Once retrieval and reranking work independently, connect them to an answer-generation model.

```
Plain text

User Question
     |
     v
Query Embedding
     |
     v
Retrieve Top 50
     |
     v
Cross-Encoder Reranking
     |
     v
Select Top 5
     |
     v
Build Context
     |
     v
Call LLM
     |
     v
Generate Answer
```

Run the same set of questions with and without reranking.

Evaluate:

- Whether the answer is factually correct.
- Whether the answer is supported by the retrieved documents.
- Whether the system misses important information.
- Whether irrelevant context causes confusion.
- Whether reranking increases latency or improves answer quality enough to justify its cost.

Do not assume that a better ranking automatically produces a better answer. The selected documents must contain the information needed to answer the question, and the LLM must use that information correctly.

---

5. Suggested project structure

Keep the code modular so that you can replace one component without rewriting the entire pipeline.

```
Plain text

rag-reranking-lab/
│
├── data/
│   ├── documents.jsonl
│   └── evaluation_queries.jsonl
│
├── src/
│   ├── document_loader.py
│   ├── chunker.py
│   ├── embedder.py
│   ├── vector_retriever.py
│   ├── bm25_retriever.py
│   ├── hybrid_retriever.py
│   ├── reranker.py
│   ├── answer_generator.py
│   └── evaluation.py
│
├── experiments/
│   ├── compare_retrievers.py
│   ├── compare_rerankers.py
│   └── compare_top_k.py
│
```

Tests I would require

Before considering the project complete, verify that:

1. The retriever returns no more than the requested number of documents.
2. The reranker returns documents sorted by descending score.
3. Every reranked document came from the original candidate set.
4. A relevant document outside the retrieved candidate set cannot appear in the reranked results.
5. Evaluation metrics match manually calculated examples.
6. An empty candidate list is handled correctly.
7. Retrieval and reranking latency are measured separately.
8. Results are reproducible enough to compare experiments meaningfully.

For a stronger evaluation, use multiple queries and labelled relevant documents rather than relying on one successful example.

---

6. Which project should you choose first?

My recommendation is to start with the **RAG Retrieval and Reranking Evaluation Lab**, using a small, controlled document collection.

You do not need to build a production-grade application immediately. You need a system that exposes the internal behaviour of each stage.

Here is the learning order I would follow:

Order	Topic	What you should be able to explain afterward
1	Text chunking	Why document boundaries affect retrieval
2	Embeddings	How text is represented as vectors
3	Cosine similarity	How initial retrieval ranks documents
4	Bi-encoder	Why queries and documents are encoded independently
5	Cross-encoder	How query-document interactions affect relevance scores
6	Reranking	How the candidate ordering changes
7	Recall and precision	How to evaluate the retriever
8	MRR and NDCG	How to evaluate the ranking quality
9	Context selection	Why top-K affects the LLM's available evidence
10	Answer evaluation	Whether retrieval improvements translate into better answers

One final recommendation: Do not begin by training your own reranker. First understand how a pretrained cross-encoder works, measure its ranking quality, and identify the situations in which it fails. Fine-tuning becomes much more meaningful once you have that baseline.

I can help you build this as a **runnable Python project, starting with Phase 1 and progressing through the full pipeline**, including the code, unit tests, labelled sample documents, and evaluation reports.

You said:

I want this using Ollama as i have ollama locally runnig. I want you to articulate the 5 phases use open source model where ever possible and strcuture the phases

ChatGPT said:

RAG Reranking Lab — 5-Phase Implementation Plan Using Ollama

I recommend building this as a **local-first RAG engineering project** using Ollama, Python, open-source models, and a local vector index. The objective is to understand the pipeline's internal workings rather than simply assemble a RAG application using a framework.

One important design decision: **use Ollama for embeddings and answer generation, but initially use a dedicated open-source cross-encoder for reranking.** This gives you a proper baseline against which you can later test an Ollama-hosted LLM as a reranker.

Ollama provides APIs for embedding generation and chat completion. The Sentence Transformers library provides pretrained cross-encoders specifically designed to score query-document pairs. GitHub +2

1. Technology stack

Component	Technology	Role
Language	Python	Implement and test every stage
Local model runtime	Ollama	Run models locally
Embedding model	`embeddinggemma`	Convert queries and document chunks into vectors
Initial vector search	NumPy cosine similarity	Understand retrieval mathematics
Vector database, later	FAISS	Experiment with efficient vector search
Keyword retrieval	BM25	Find documents using lexical matching
Reranker	`cross-encoder/ms-marco-MiniLM-L6-v2`	Score query-document pairs
Local answer-generation model	Any suitable instruct model already available in Ollama	Generate answers using retrieved context
Evaluation	Python, pandas, NumPy	Measure retrieval and ranking quality
Optional evaluation framework	Ragas	Evaluate RAG quality in later phases

Ollama documents `embeddinggemma` as an embedding model for semantic search and RAG. You can explore its embedding documentation. GitHub

Why not use Ollama for every component immediately?

Because your primary learning objective is reranking.

A dedicated cross-encoder gives you a clear model architecture to investigate: it receives the query and a candidate document together and produces a relevance score. It is different from asking a general-purpose LLM to read a list of documents and sort them.

The proposed cross-encoder is an open model available through the Sentence Transformers ecosystem. Its model weights are downloaded from Hugging Face and run locally; it does not require a paid API. Sbert +1

Local execution note: The embedding and generation stages can run through Ollama. The initial cross-encoder runs through Python and PyTorch, not Ollama, so you will need to install the relevant Python dependencies. Later, you can compare it with an LLM-based reranker served by Ollama.

Phase 1 — Build the Retrieval Engine from Scratch

Objective: Understand how documents are chunked, embedded, indexed, and retrieved before introducing reranking.

Estimated effort: 3–4 days.

1.1 What you will implement

Build the first pipeline without LangChain or LlamaIndex.

Plain text

```
Documents
    |
    v
Document Loader
    |
    v
Text Chunker
    |
    v
Ollama Embedding Model
    |
    v
Document Vectors
    |
    v
Cosine Similarity Search
    |
    v
Top-K Retrieved Documents
```

Start with approximately 100–500 document chunks about RAG, embeddings, vector databases, and Transformer architectures.

Create a small knowledge base that includes:

- Relevant documents.
- Semantically similar but irrelevant documents.
- Documents that share keywords but answer different questions.
- Documents containing the answer in different wording.
- Documents that contain only part of the answer.

These variations are important because they expose the weaknesses of retrieval.

1.2 Set up Ollama

If Ollama is already running, check the available models:

Bash

```
ollama list
```

Download an embedding model:

Bash

```
ollama pull embeddinggemma
```

Install the Python dependencies:

Bash

```
pip install ollama numpy pandas
```

Use Ollama's embedding API to generate document vectors. For example:

Python

```
import ollama

response = ollama.embed(
    model="embeddinggemma",
    input=[
        "A bi-encoder independently encodes queries and documents.",
        "A cross-encoder jointly processes queries and documents."
    ]
)
```

```
vectors = response["embeddings"]

print("Number of vectors:", len(vectors))
print("Vector dimensions:", len(vectors[0]))
```

This lets you inspect the actual numerical representations produced by your local model.

Next, implement cosine similarity yourself using NumPy. Compare the resulting ranking with your expectations.

1.3 Experiments

Experiment	Change	What you learn
E1	Change the user query	How query wording affects retrieval
E2	Change chunk size	How chunk boundaries affect relevance
E3	Change chunk overlap	Whether relevant context gets split across chunks
E4	Compare cosine scores	How the retriever orders documents
E5	Compare keyword and semantic matches	Why semantic retrieval can find documents without exact keyword matches

1.4 Deliverable

Create `phase1_retrieval.py`.

It should print:

- The original query.
- The retrieved document IDs.
- Each document's similarity score.
- The ranked document contents.
- The time taken to retrieve results.

Success criterion: You can explain why a particular document appears at rank 1 and why another document appears at rank 20.

Do not proceed until you can reproduce the ranking and calculate cosine similarity independently.

Phase 2 — Implement a Real Cross-Encoder Reranker

Objective: Understand how reranking changes the ordering of retrieved documents.

Estimated effort: 3–4 days.

This is the most important phase of your project.

2.1 Architecture

Plain text

```
User Query
    |
    v
Phase 1 Retriever
    |
    v
Retrieve Top 50 Documents
    |
    v
Cross-Encoder Reranker
    |
    +-- Query + Document 1 --> Score 1
    |
    +-- Query + Document 2 --> Score 2
    |
```

```
    +-- Query + Document 3 --> Score 3
    |
    +-- ... 47 more pairs
    |
    v
Sort by Reranker Score
    |
```

2.2 Model selection

Use:

`cross-encoder/ms-marco-MiniLM-L6-v2`

This model is a good starting point because it is specifically trained for passage relevance ranking. Its architecture and usage are documented by Sentence Transformers. Sbert

Install the dependencies:

Bash

```
pip install sentence-transformers torch
```

Create `phase2_reranking.py`.

The implementation should:

1. Receive the query.
2. Receive the 50 retrieved documents from Phase 1.
3. Construct 50 query-document pairs.
4. Pass those pairs to the cross-encoder.
5. Capture the score for every document.
6. Sort documents by descending score.
7. Return the top 5.
8. Record the ranking before and after reranking.

For each query-document pair, the model produces a relevance score. The score is used to order candidates; it should not automatically be interpreted as a probability.

The cross-encoder evaluates each pair jointly, which allows it to model interactions between the query and document tokens. Unlike a bi-encoder, it does not independently generate a reusable embedding for each document. Sbert +1

2.3 Experiments

Run these experiments using exactly the same retrieved candidate set.

Experiment	Configuration	Question
E1	Initial retrieval only	What is the original ranking?
E2	Retrieve 20, rerank 20	How much does reranking help with a small candidate set?
E3	Retrieve 50, rerank 50	Does a larger candidate set help?
E4	Retrieve 100, rerank 100	Does the improvement justify the additional latency?
E5	Change the query wording	How sensitive is the ranking to query formulation?
E6	Include irrelevant documents	Can the reranker push unrelated candidates down?

2.4 Deliverable

Create `phase2_reranking.py` and generate a CSV report containing:

Plain text

```
query_id
document_id
initial_rank
initial_similarity_score
rerank_score
reranked_position
is_relevant
```

You should be able to compare a document's initial position with its reranked position.

Success criterion: You can explain why a document moved from rank 35 to rank 2, inspect its relevance score, and verify whether that change improved the ranking of relevant documents.

One limitation to investigate: the reranker cannot recover relevant documents that the retriever failed to include in its candidate set.

Phase 3 — Evaluate Retrieval and Reranking Scientifically

Objective: Prove whether reranking improves retrieval quality rather than relying on a few examples.

Estimated effort: 4–5 days.

This phase turns your implementation into a genuine RAG evaluation laboratory.

3.1 Create a labelled evaluation dataset

Create at least 30–50 questions.

For each question, identify the relevant document IDs manually.

Example:

JSON

```
{
  "query_id": "Q001",
  "query": "How does a cross-encoder score documents?",
  "relevant_documents": ["D012", "D034"]
}
```

Start with a manageable dataset, but ensure it includes difficult cases.

For example:

- Questions where the answer uses different terminology from the query.
- Questions with multiple relevant documents.
- Questions where several documents discuss the same topic but answer different questions.
- Questions for which no document contains the answer.

Manually labelled relevance is your ground truth. Do not use the reranker's own scores as the relevance labels.

3.2 Implement the evaluation metrics

Metric	Purpose	What you want to understand
Recall@5	Relevant documents in the first five results	Whether important documents were retrieved
Recall@50	Relevant documents in the first 50 results	Whether the initial retrieval stage found the candidates
Precision@5	Proportion of the first five results that are relevant	Whether the selected context is dominated by relevant material
MRR@10	Position of the first relevant result	How quickly a user gets a useful document

Metric	Purpose	What you want to understand
NDCG@5	Ranking quality, accounting for relevance and position	Whether the most relevant documents are ranked highest
Latency	Processing time	Whether improved ranking comes at an acceptable cost

For MRR, if the first relevant document appears at rank 4, the reciprocal rank for that query is:

$$RR = \frac{1}{4} = 0.25$$

MRR is the average reciprocal rank across your evaluation queries.

For NDCG, use graded relevance labels if you want to distinguish highly relevant documents from partially relevant ones.

3.3 Compare the results

Run two experiments:

Baseline

```
Query
  |
  v
Retrieve Top 50
  |
  v
Select Top 5
```

Reranked

```
Query
  |
  v
Retrieve Top 50
  |
  v
Rerank Top 50
  |
  v
Select Top 5
```

Use the same queries, documents, relevance labels, and initial retrieval results in both experiments.

Produce a comparison report:

Metric	Baseline	Reranked
Recall@5	Measure	Measure
Precision@5	Measure	Measure
MRR@10	Measure	Measure
NDCG@5	Measure	Measure
Average latency	Measure	Measure

These are placeholders, not expected results. You must calculate them using your own dataset.

Important distinction

If reranking improves NDCG@5 but Recall@50 remains unchanged, that is expected: reranking changes the ordering of the retrieved candidates, not the contents of the original candidate set.

If Recall@5 improves, that means relevant documents have moved into the top five. It does not mean that the reranker has discovered documents that the initial retrieval stage missed.

3.4 Deliverable

Create:

- `phase3_evaluation.py`
- `evaluation_queries.jsonl`
- `baseline_results.csv`
- `reranked_results.csv`
- `evaluation_summary.csv`

Success criterion: You can demonstrate with measured results whether reranking improves your chosen ranking metrics, and explain the cases where it does not.

Phase 4 — Compare Open-Source Retrieval and Reranking Strategies

Objective: Understand the trade-offs involved in designing a production RAG retrieval pipeline.

Estimated effort: 4–5 days.

By now, you will understand the basic retriever and cross-encoder. This phase introduces alternatives.

4.1 Compare different retrieval approaches

Implement the following configurations:

Experiment	Initial retrieval	Reranker
A	Dense retrieval using Ollama embeddings	None
B	BM25 keyword retrieval	None
C	Dense retrieval	Cross-encoder
D	BM25 + dense hybrid retrieval	None
E	BM25 + dense hybrid retrieval	Cross-encoder

Use a library such as `rank-bm25` for keyword retrieval:

```bash
pip install rank-bm25
```

Use your existing NumPy implementation for dense retrieval initially. Introduce FAISS once you want to investigate more efficient vector indexing.

For hybrid retrieval, combine the rankings using Reciprocal Rank Fusion (RRF), rather than directly adding incomparable raw scores from BM25 and cosine similarity.

4.2 Add an Ollama-based LLM reranker

This is where your local Ollama setup becomes particularly useful.

You will compare two approaches:

Approach A — Dedicated cross-encoder

```plain text
Query + Document
       |
```

```
      v
Cross-Encoder
      |
      v
Relevance Score
```

Approach B — LLM-based reranking

Plain text

```
Query + Candidate Documents
      |
      v
Local Ollama Instruct Model
      |
      v
Structured Relevance Judgments
      |
      v
Sort Documents by Relevance
```

For Approach B, use an instruction model already available in your Ollama installation. If you need a small model, you can consider a model such as `qwen2.5:3b`, subject to your hardware and the model's availability in your Ollama environment.

Ask the LLM to evaluate each document against the query and return a structured relevance judgment. Use a consistent scoring rubric and parse the response programmatically.

For a fair comparison, run both rerankers against the same candidate set and labelled questions.

Important: An LLM-based reranker is not equivalent to a dedicated cross-encoder. It may require more inference time and may produce inconsistent scores. Treat this as an experiment, not an assumption that the LLM will perform better.

4.3 Experiments

Measure:

- NDCG@5 and MRR@10.
- Recall@50 before reranking.
- Reranking latency.
- Memory usage.
- Number of model evaluations.
- Ranking consistency across repeated runs, especially for LLM-based scoring.

4.4 Deliverable

Create `phase4_retrieval_comparison.py`.

The output should compare all five retrieval configurations and the two reranking approaches.

Success criterion: You can explain why a system might choose hybrid retrieval with a cross-encoder instead of dense retrieval alone or LLM-based reranking.

Phase 5 — Build a Complete Local RAG Application Using Ollama

Objective: Connect all five major components into a working, evaluated RAG system.

Estimated effort: 4–5 days.

This phase builds on everything you have learned.

5.1 Final architecture

Plain text

```
                USER QUESTION
                      |
                      v
              Query Processing
```

```
                |
                v
        +----------------------+
        | Retrieval Layer      |
        |                      |
        | Dense / BM25 / Hybrid|
        +----------------------+
                |
                v
          Retrieve Top 50
                |
                v
        +----------------------+
        | Reranking Layer      |
        |                      |
        | Cross-Encoder        |
        | OR Ollama LLM        |
```

5.2 Generate the answer with Ollama

Use the chat model you already have locally.

Your answer-generation component should:

1. Receive the user's query.
2. Receive the selected document chunks.
3. Construct a prompt containing the retrieved context.
4. Call the Ollama chat API.
5. Return an answer with references to the source document IDs.
6. Avoid inventing an answer when the context does not support one.

For example, you can instruct the model:

Plain text

```
You are a question-answering assistant.

Answer the question using only the supplied context.

If the context does not contain sufficient information,
state that the answer cannot be determined from the
available documents.

Cite the document IDs supporting your answer.

Question:
{query}

Retrieved Context:
{context}
```

Use Ollama's API documentation to integrate the chat and embedding calls. GitHub +1

5.3 Evaluate the final answers

Run each test question twice:

- Once using the baseline retrieval results.
- Once using the reranked results.

Compare the answers on:

Evaluation dimension	Question
Correctness	Does the answer correctly address the query?
Faithfulness	Is the answer supported by the retrieved documents?
Context relevance	Did the selected documents contain the necessary information?
Citation accuracy	Do the cited document IDs support the claims?
Latency	How much time does each pipeline take?

You can initially assess correctness and citations manually. Later, introduce Ragas or another open-source evaluation tool to automate parts of the assessment.

Do not assume that better retrieval metrics guarantee better answers. The final evaluation must establish whether the improved context actually helps the LLM.

5.4 Final deliverables

Your completed project should contain:

Plain text

```
rag-reranking-lab/
│
├── data/
│   ├── documents.jsonl
│   └── evaluation_queries.jsonl
│
├── src/
│   ├── document_loader.py
│   ├── chunker.py
│   ├── ollama_embedder.py
│   ├── dense_retriever.py
│   ├── bm25_retriever.py
│   ├── hybrid_retriever.py
│   ├── cross_encoder_reranker.py
│   ├── ollama_llm_reranker.py
│   ├── ollama_generator.py
│   └── evaluation.py
│
├── experiments/
│   ├── phase1_retrieval.py
│   ├── phase2_reranking.py
│   └── phase3_evaluation.py
```

Success criterion: You can run the entire pipeline locally, compare different retrieval and reranking configurations, and explain which configuration works best for your labelled evaluation dataset.

---

7. Recommended implementation schedule

Week	Phase	Primary outcome
Week 1	Phase 1	Working local embeddings and retriever
Week 2	Phase 2	Cross-encoder reranking and before/after rankings
Week 3	Phase 3	Evaluation metrics and reproducible comparisons
Week 4	Phase 4	Dense, BM25, hybrid, and LLM reranking experiments
Week 5	Phase 5	Complete local RAG application with answer evaluation

This schedule assumes you can spend a few hours a day on the project. You can shorten it by keeping the initial dataset small and postponing optional frameworks.

My final recommendation

Build this in two distinct layers:

Layer 1: Core RAG engineering

- Ollama embeddings.
- NumPy-based retrieval.
- Cross-encoder reranking.
- Evaluation metrics.

Layer 2: Production-oriented extensions

- FAISS.
- BM25 and hybrid retrieval.
- Ollama LLM-based reranking.

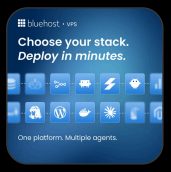
- Answer generation and RAG evaluation.

This separation is valuable because it allows you to establish a measurable baseline before introducing additional complexity.

One practical consideration: the embedding and generation models run locally through Ollama, but the dedicated cross-encoder initially runs through Python and PyTorch. You can keep the entire application local after downloading the required models.

The best next step is to implement Phase 1 completely, including the folder structure, sample documents, Ollama embedding integration, cosine similarity implementation, and unit tests. Then use its output as the input to Phase 2, so you can observe the reranker changing a real retrieval ranking rather than relying on hypothetical scores.

This might also interest you.



Bluehost Ad

Deploy AI agents. Host anything. Scale instantly.

Run Claude Code, n8n, OpenClaw or Hermes in an always-on environment with full root access.

Learn more